

FlexServer: A High-Performance Content Server

Leveraging Asynchronous I/O and SmartNICs

Abstract—We present FlexServer, a high-performance content server designed to optimize efficiency by effectively minimizing the overhead of disk read operations and packet transfer operations. For this, we leverage a SmartNIC and the modern asynchronous I/O mechanism, *io_uring*. To address the inefficiency caused by duplicate transmissions of popular content, we integrate a caching mechanism directly on the SmartNIC. The control plane employs a multi-type buffer-based consistency management algorithm, ensuring full consistency with TCP. FlexServer is implemented using the mTCP user-level TCP stack and an NVIDIA BlueField-3 SmartNIC equipped with 400 Gbps Ethernet ports. Our evaluations show that FlexServer achieves a throughput of up to 199 Gbps with a single host CPU core and 351 Gbps with 8 cores, significantly outperforming the state-of-the-art IO-TCP ($2.5\times - 4.4\times$), which delivers only 80 Gbps in both cases under typical workload scenarios.

Index Terms—content server, SmartNIC, high-performance.

I. INTRODUCTION

According to Sandvine’s 2023 Global Internet Phenomena Report [1], video traffic accounted for 65% of all Internet traffic in 2022, highlighting its dominant role in today’s Internet usage. The report further indicates that applications such as marketplace platforms, gaming, social networking, cloud services, web browsing, and file-sharing collectively followed video services in terms of total traffic volume. Many of these applications rely on content servers to deliver large volumes of static content to users. For service providers catering to a substantial user base, establishing an efficient content delivery system is essential to enhance user experience and minimize operational costs.

A significant challenge in implementing high-performance content servers lies in the growing disparity between the advancements of CPUs and I/O devices such as NVMe SSDs and NICs, which directly impacts the server’s ability to efficiently handle content delivery. Content servers must load content into main memory before delivering it to users. This process consumes a significant number of CPU cycles, and the memory bandwidth often becomes a critical performance bottleneck.

In response to this challenge, many research efforts focused on improving the performance of content servers by reducing this inefficiency. For example, the Atlas server [2] avoids the inefficiencies of the traditional disk buffer cache by leveraging DDIO (Direct Data I/O) technology [3]. This approach processes video chunks in the Last Level Cache (LLC) by directly transferring data from disks into the LLC via DMA, encrypting the data in place, packetizing it, and transferring it to the NIC without involving RAM. Similarly, IO-TCP [4] introduces a content server architecture that offloads data-plane operations, such as disk I/O and packet transfers, to SmartNICs while retaining the control plane operations on the host CPU. Even though these solutions mitigate inefficiencies and improve throughput performance, none have achieved throughput close to the line rate of modern high-performance NICs (e.g., 400 Gbps) with full TCP support.

To bridge this gap, we focus on maximizing the throughput performance of content servers using 400 Gbps SmartNICs. In this effort, we identify three challenges.

- **Overhead from POSIX file read operations:** Despite the efforts of previous work, the file read operations continue to impose significant overhead. For example, in IO-TCP, the high number of disk I/O operations causes the memory bandwidth in SmartNICs to become a performance bottleneck, limiting the effective utilization of both host CPU and SmartNIC resources.
- **Inefficiency from lack of caching:** While the I/O offloading approach used in state-of-the-art solutions like IO-TCP significantly improves throughput, it remains inefficient due to the absence of caching mechanisms. Although traditional OS disk buffer caches are often suboptimal, reducing duplicate I/O operations for popular content could enhance performance in many service scenarios.
- **Maintaining TCP consistency:** Ensuring TCP consistency is a critical challenge when multiple objects within a single flow are transmitted across different devices (i.e., host and SmartNIC) using various methods (e.g., file reads and caching). Seamless operation under these conditions is essential for reliable, high-performance content delivery.

To tackle these challenges, we introduce FlexServer, a SmartNIC-based high-performance content server. FlexServer optimally distributes the packet transfer operations between the host and SmartNIC to fully utilize resources of both. In addition, to reduce the overhead of file read operations efficiently, FlexServer leverages modern asynchronous I/O mechanisms provided by *io_uring*, enabling it to hide disk read latencies and utilize CPU cycles more effectively. To mitigate the inefficiency of duplicate transmissions of popular content, FlexServer incorporates a custom caching mechanism directly on the SmartNIC. Furthermore, to ensure complete consistency with TCP when multiple objects within a single flow are processed through different data paths, FlexServer employs a novel buffer management algorithm in the control plane, providing reliable and efficient operation.

We implement FlexServer using an NVIDIA Bluefield-3 SmartNIC with 400 Gbps Ethernet ports. For the host’s TCP stack, we utilize mTCP [5], a user-level TCP implementation, and develop a simple API to allow applications to utilize FlexServer’s capabilities with minimal code modifications.

Through extensive evaluations, we show that FlexServer can significantly outperform the state-of-the-art SmartNIC solution, IO-TCP. Even using only a single host CPU core, FlexServer’s read operations achieve 199 Gbps throughput exceeding IO-TCP’s maximum throughput of approximately 80 Gbps. With 8 host CPU cores, FlexServer reaches 351 Gbps, achieving $1.6\times$, $5.1\times$, and $4.4\times$ higher throughput compared to Linux, mTCP, and IO-TCP, respectively.

The main contributions of this work are as follows.

- We identify key performance enhancement opportunities of existing content servers by leveraging up-to-date asynchronous I/O mechanisms, custom caching strategies, and SmartNICs.
- We present FlexServer, a high-performance content server architecture that can fully utilize the SmartNIC and host resources by

optimally distributing different types of transmission operations between them.

- We validate that FlexServer achieves significantly higher throughput than existing solutions under typical workload scenarios.

II. BACKGROUND & MOTIVATION

A. Limitations of Existing High-Performance Content Servers

Modern content delivery heavily relies on Content Delivery Networks (CDNs), which are composed of geographically distributed content servers. These servers store and deliver various types of web content, including HTML files, images, audio, video, and applications, ensuring low-latency access for users by being physically closer to them [6]. The design of content servers has focused on optimizing disk I/O and CPU utilization. Techniques such as *sendfile()* are widely used to reduce memory copying and context-switching overheads. Additionally, content servers typically employ event-driven architecture [7], [8], which utilizes system calls like *select()* to handle events across multiple connections concurrently. When a system call completes, event handlers process ready socket descriptors by accepting new connections or performing I/O operations for connections ready to read or write. With advancements in hardware, such as large-capacity RAM and SSDs, the primary bottleneck has shifted to the memory subsystem [2]. Multiple memory copies during disk and network I/O operations, combined with encryption and decryption processes, exacerbate this bottleneck, presenting challenges for optimizing modern servers [4].

To overcome these bottlenecks and enhance content server performance, IO-TCP [4] utilized SmartNICs by offloading data-plane operations such as disk I/O, encryption, and network I/O to the SmartNICs, while retaining control-plane operations on the host CPU. However, IO-TCP has several limitations. Its reliance on POSIX *read()* for disk I/O is inefficient, consuming substantial ARM core cycles and incurring additional overhead from interrupts and context switching. Moreover, the static assignment of I/O operations to the SmartNIC leads to underutilization of host CPU cores. While IO-TCP bypasses the operating system's file buffer cache using direct I/O, it sacrifices the advantages of caching frequently accessed content. These constraints prevent IO-TCP from achieving the high throughput required to fully utilize modern high-performance NICs.

B. Recent Advancement in Disk I/O Mechanisms

Linux provides two widely used APIs for file I/O operations; the POSIX read/write file I/O APIs for synchronous operations and the POSIX asynchronous I/O (AIO) APIs, implemented as *libaio* [9]. Both APIs incur significant overhead due to their reliance on systems calls for kernel interactions. A more recent advanced API *io_uring* [10] was introduced in Linux kernel version 5.1 and has demonstrated higher performance and better CPU utilizations [11].

io_uring provides two queues, submission queues (SQs) and completion queues (CQs), which streamline the interaction between applications and the kernel for storage operations [12]. Applications submit I/O requests to the SQs and receive notifications about completed operations via the CQs. By default, *io_uring* uses *io_uring_enter* system call for both submitting requests and retrieving completion events. To further reduce overhead, it supports two advanced modes: completion polling mode, which eliminates system calls for processing completions, and submission polling mode, which removes system calls for request submissions.

C. Streamlining the Data Plane Using SmartNICs

A SmartNIC is a network interface card enhanced with additional resources such as CPUs, NPUs (Network Processing Units), memory, accelerators, or FPGAs. These capabilities allow the NIC to execute user-programmed tasks, offloading jobs typically handled by the host CPU. SmartNICs have been used to accelerate various systems, including distributed applications [13], serverless computing systems [14], key-value stores [15], [16], mobile communication networks [17], [18], distributed storage systems [19], [20] and TCP processing [4], [21], [22].

SmartNICs can significantly enhance the data plane of content servers by caching popular content in their memory and offloading data-plane operations. However, achieving maximum throughput in high-performance NIC-based content servers presents two key challenges:

- **Limited SmartNIC resources:** SmartNICs typically feature low-clock-speed ARM CPUs with smaller cache capacities compared to host CPUs. Consequently, the cycle budget per SmartNIC core is limited, requiring efficient use of processing cycles per packet. Additionally, SmartNICs often have restricted memory capacity and lower memory bandwidth. These constraints necessitate an optimal division of labor between the host and SmartNIC to achieve high performance.
- **Coordination between the host and SmartNIC:** In systems where request processing is dynamically distributed between the host and SmartNIC, maintaining TCP semantics can be challenging. For example, data flows may include mixed formats, such as general data, file data, and cached data (e.g., in HTTP/2 multiplexing). To ensure correct TCP behavior, tight coordination is essential. This includes managing TCP sequence numbers accurately for all data bytes, even in complex scenarios like packet losses and retransmissions.

III. DESIGN OF FLEXSERVER

In this section, we provide an overview of the design and implementation of the proposed system, FlexServer, detailing its key features and innovative components.

A. Design Principles

We present FlexServer, a novel content server architecture designed to leverage modern asynchronous I/O abstractions and SmartNICs to accelerate the data plane, achieving maximum throughput and efficient resource utilization. Its design is guided by the following principles:

- **Efficient division of control and data planes.** FlexServer executes the control plane operations on the host, utilizing its high-speed CPUs for complex control operations, while dedicating remaining host and SmartNIC resources to the data plane, thereby optimizing performance.
- **Hiding disk read latency through asynchronous disk I/O.** FlexServer integrates Linux's advanced asynchronous I/O abstraction, *io_uring*, into the host's user-level network stack, enabling the system to minimize disk read overhead and resource consumption by hiding latency through asynchronous operations.
- **SmartNIC-based caching mechanism.** Instead of relying on the traditional disk buffer cache, which incurs high overhead when reading new data from disks, FlexServer implements a custom caching mechanism on SmartNICs to reduce the overhead of reading new data from disks and accelerates access to frequently used content.

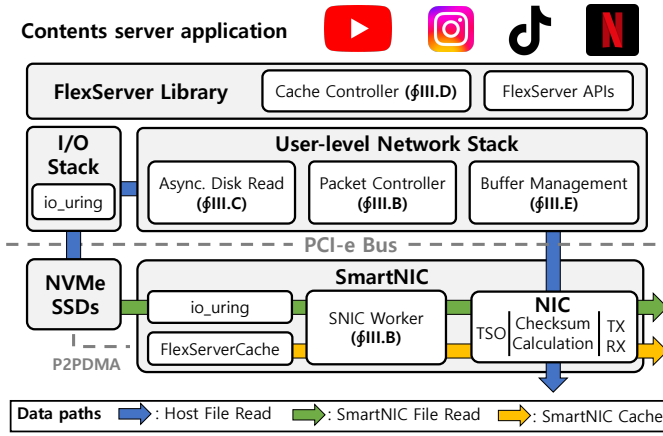


Figure 1: Architecture of FlexServer. FlexServer provides three distinct data paths for streaming TCP flows: 1. Host File Read, 2. SmartNIC Cache, and 3. SmartNIC File Read.

- **Multi-type buffer-based TCP consistency management.** To maintain TCP consistency across diverse content types—including general data, file data, and cached data—FlexServer employs a multi-type buffer mechanism in its control plane.

B. Division of Control and Data Planes Across Host and SmartNIC

Unlike traditional content servers that process TCP streams exclusively on the host, FlexServer introduces a tripartite data path leveraging SmartNIC capabilities, as illustrated in Figure 1. This is achieved through novel components on both the host and SmartNIC, with the control functionality abstracted into the *libflexserver* API, which operates in the host’s userspace and interacts with host and SmartNIC components.

Host operations. The Cache Controller dynamically manages the SmartNIC cache, storing frequently accessed content based on popularity metrics. The Packet Controller constructs TCP packets with the object’s metadata as the payload and determines the transmission path. If the object is designated for the SmartNIC, the Packet Controller sends the metadata to the SmartNIC, which either caches the data or reads the object for transmission. If the object is assigned to the host, the host retrieves it using asynchronous disk reads facilitated by the Asynchronous Disk Read module, constructs the TCP packet, and sends it to the NIC. The NIC then performs TCP Segmentation Offload (TSO) and checksum calculations before transmitting the packet over the network. The Buffer Management module ensures TCP consistency by using a multi-type buffer to organize metadata for objects of different types (e.g., host file reads, SmartNIC cache operations, and SmartNIC file reads).

SmartNIC operations. The SmartNIC operates by executing commands received from the host, such as OFFLOAD, EVICT, TX_CACHE, and TX_DISK, as detailed in Table I. Since the SmartNIC does not maintain connection state, the system avoids the overhead of synchronizing connection states between the host and SmartNIC. Each core of the SmartNIC executes identical, straightforward operations based solely on the commands issued by the host, ensuring efficient and parallel processing.

C. Hiding the Disk Read Latency Using Asynchronous Disk I/O

To achieve a high-performance content server, optimizing disk I/O performance is critical. However, both conventional Linux systems [23] and recent research [4] still face challenges with disk read

Table I: FlexServer Commands Used in SmartNIC Operations

Command	Description
OFFLOAD	Add a file to the SmartNIC cache.
EVICT	Remove less popular objects from the SmartNIC cache.
TX_CACHE	Transmit data using the cached object in SmartNIC memory, replacing the payload with the cached data.
TX_DISK	Read the requested file directly from storage and transmit it by replacing the payload with the file data.

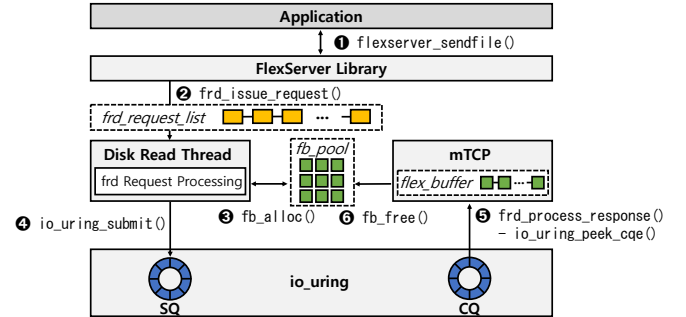


Figure 2: Operation of asynchronous file read using *io_uring* in FlexServer.

operations due to the limitations of blocking I/O. FlexServer addresses this challenge by seamlessly integrating a novel asynchronous read operation into the user-level network stack, mTCP.

Asynchronous file read operation. Figure 2 illustrates the operation of FlexServer’s asynchronous file read mechanism, which utilizes *io_uring*. To support this, we introduce a Disk Read Thread alongside mTCP’s packet processing thread. The process begins with ① the application invoking *flexserver_sendfile()* from the FlexServer Library (*libflexserver*). In response, ② mTCP places the file read request into the file read request list (*frd_request_list*) and immediately returns without waiting for the request’s completion. The Disk Read Thread then retrieves the request from the list and ③ allocates the file read buffers. Next, ④ it submits the file I/O request to the *io_uring* submission queue (SQ). Once the disk read operation completes, *io_uring* places the completion information into the completion queue (CQ). Subsequently, ⑤ mTCP retrieves this information, updates the multi-type buffer information of the flow that issued the request, and activates the flow by enqueueing it into the mTCP FIFO queue. While processing the packets of the activated flow, mTCP sends the data read through *io_uring*. Finally, ⑥ mTCP releases the file read buffers for the processed flow, making them available for reuse by other flows. By returning immediately without waiting for read completion, *flexserver_sendfile()* allows the application to handle other file transmission requests concurrently, thereby avoiding head-of-line blocking.

Performance optimization of asynchronous file read operations. In mTCP, the application thread and the packet processing thread compete for CPU resources. In our system, the addition of a disk read thread introduces a third competing thread. To ensure disk read throughput, we employ request batching. The disk read thread processes requests from the file read request list when the list size exceeds a threshold (16 in our system) or when a timer expires (50 microseconds). It then submits batched file I/O requests to the *io_uring* SQ.

To further optimize performance, we leverage Scatter-Gather DMA (SG-DMA) for packet transmission. DPDK supports external buffers [24], which abstract SG-DMA functionality, allowing DMA operations on multiple non-contiguous memory regions. By config-

using the IOVA for SG-DMA to directly reference allocated blocks of the file read buffer, we eliminate memory copy overhead in packet transmission. Additionally, we enhance mTCP by integrating TSO.

D. Caching Using the SmartNIC Memory

Cache data structures. FlexServer introduces a caching mechanism for popular objects using SmartNIC memory. The cache is divided into multiple fixed-size blocks, where a single object may occupy one or more blocks. This design, while potentially causing memory inefficiency due to internal fragmentation, prioritizes performance by eliminating the need for defragmentation and enabling efficient DMA operations through memory alignment.

Caching algorithm. The caching algorithm manages data between the SmartNIC memory and disk. It operates every R request and performs the following steps. It first selects *demotion candidates* from the SmartNIC cache by randomly sampling D_c objects. These candidates are ranked based on their priorities, and the D_f objects with the lowest priorities are chosen for demotion. After that, it selects *promotion candidates* from the disk by randomly sampling P_c objects. From these, P_f objects with the highest priorities are promoted, ensuring their combined size fits within the total number of blocks freed by the demoted objects.

This approach resembles the algorithm in [25]. The priority of an object i is calculated using the following equation.

$$p_i \leftarrow (1 - \omega) \frac{n_i}{s_i t_i} + \omega p_i. \quad (1)$$

where n_i , s_i , and t_i , are the number of requests, the object size, and the time from the previously sampled moment for object i respectively. Let $\omega \in [0, 1]$ be a balancing parameter.

Dedicated offloading command queues and batching. Using a shared queue for file transmissions and offloading commands can lead to head-of-line blocking. To address this, FlexServer assigns a dedicated queue for offloading commands. Additionally, commands are batched into a single packet within a time threshold to minimize overhead and enhance performance.

E. TCP Consistency Management in the Control Plane

To maintain TCP consistency in our system, where different data paths can be utilized depending on the cache and resource status, we propose a multi-type buffer-based mechanism. This buffer stores metadata for files transmitted via different data paths, including a file read from the disk at the host, an object cached in the SmartNIC memory, and a file read from SmartNIC. The control plane at the host constructs TCP packets depending on each data type using the multi-type buffer, and the corresponding data plane is in charge of filling the packet with the object and transmitting the finally reconstructed packet. The control plane considers the object size to be sent at different data planes and adjusts the TCP sequence numbers appropriately. The advantages of this mechanism are twofold: first, the data plane operates without maintaining state for each flow, which simplifies the overall system structure and reduces operational complexity; second, the host can implement this mechanism with minimal modifications to the application and TCP code.

Multi-type buffer example. Figure 3 shows an example of the control plane operations for a single flow using multi-type buffers. Each entry in the multi-type buffer contains information about the files to be sent from the host and SmartNIC, including the size of the file ($data_len$), its location in memory of either the host or SmartNIC ($memory_offset$), the TCP sequence number for the first byte of the file ($head_seq$), and the length of the successfully transmitted part ($already_sent$). The $memory_offset$ is either the location of

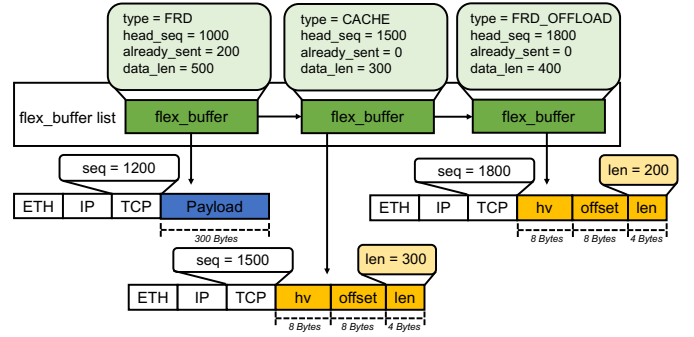


Figure 3: Example of the control plane operations using multi-type buffers.

the assigned file read buffer or the cache block. The $head_seq$ is determined by considering the initial TCP sequence number of the flow and the length of the general payload that can be sent before each file. Our scheme is capable of handling scenarios where multiple headers and objects are transmitted within a single stream, such as in HTTP/2 multiplexing.

Figure 3 illustrates a scenario where 200 bytes from the first entry have been sent and acknowledged, and 800 bytes are ready to transmit due to an expanded congestion window. The control plane first creates a TCP packet with sequence number 1200 (snd_next) and fills it with 300 bytes from the file read buffer, sending it to the DPDK buffer.

Next, it generates a packet for the cached object on the SmartNIC, including the hash value, cache block location, and data length, and places it in the DPDK buffer. A third packet is then created for the file object to be sent from the SmartNIC, containing its hash, file read buffer block location, and 200 bytes of data.

Upon receiving an acknowledgment (ACK), the control plane sequentially processes the multi-type buffer. Fully acknowledged entries are removed, while unacknowledged entries are updated by adjusting the $already_sent$ field based on the difference between ack_seq and $head_seq$.

F. Load Balancing across Three Data Paths

We propose a load-balancing mechanism to distribute workloads across three data paths: Host File Read, SmartNIC Cache, and SmartNIC File Read. The primary goal of this mechanism is to maximize the utilization of SmartNIC resources. Since caching is inherently more efficient than file offloading, the mechanism prioritizes data transmission through the SmartNIC Cache when cached file objects are available. Remaining resources are then allocated to the SmartNIC File Read path.

Implementing this mechanism requires determining the capacity of the SmartNIC File Read data path. This capacity is empirically measured by analyzing the maximum throughput achievable on the SmartNIC File Read path while adjusting the throughput for the SmartNIC Cache path. These empirical measurements are used to dynamically allocate requests to the SmartNIC, ensuring optimal resource utilization and maintaining high performance.

G. Implementation Details

The FlexServer library at the host is implemented in C, with 5,049 LOC dedicated to the Cache Controller and 626 LOC for its APIs. For the user-level network stack, we use mTCP version 3, modified with 11,345 additional LOC. As the application, we adapt the Lighttpd web server [26], requiring 822 LOC modifications. The SNIC Worker is implemented as a DPDK application, comprising 5,853 LOC.

Packets sent from the host are distributed to SNIC Worker threads, each bound to a dedicated core, via Receive Side Scaling (RSS) using 4-tuple hashing. We implement a memory pool for the SmartNIC cache and a hash table to locate cached objects efficiently. Each SNIC Worker thread maintains its own aligned file read buffer pool for file reads. When the host sends an offloading command for a file read, the SNIC Worker performs the file read using the *io_uring* API. Similar to the host, SG-DMA is employed to reduce memory copy overhead during packet transmission.

The FlexServer library provides APIs to facilitate resource management and data transmission. Key functions include *flexserver_initialize()* and *flexserver_finalize()* for allocating and releasing resources for each library instance, *flexserver_is_cached()* for checking whether an object resides in the cache, and *flexserver_sendfile()* and *flexserver_sendcache()* for transmitting file and cached objects, respectively. The *flexserver_sendfile()* function reads a file from disk using an asynchronous, non-blocking method, either on the host or SmartNIC, depending on the SmartNIC's status, and handles its transmission.

IV. EVALUATION

We evaluate FlexServer with the following research questions. (1) How much performance improvement does FlexServer provide compared to existing representative solutions? (2) How do different components or design decisions of FlexServer impact performance? (3) Does FlexServer achieve consistent performances across various hardware configurations and content request patterns?

A. Experiment Setup

Our testbed consists of one server and one client machine, both equipped with BlueField 3 SmartNICs [27]. The server is configured with an Intel Xeon w5-3433 CPU, 256 GB DDR 5 memory, and 10 Seagate FireCuda-530 1 TB SSDs for content storage. The server runs Ubuntu 20.04.6 LTS with a 5.13 kernel. The client is similarly equipped with an Intel Xeon w5-3433 CPU and 128 GB DDR5 memory, running Ubuntu 20.04.6 LTS with a 5.15 kernel.

The BlueField 3 SmartNICs feature two 400 GbE/NDR InfiniBand ports, 16 ARM cores, and 32 GB of onboard memory. We use DOCA version 2.5.2 to program the SmartNICs. At the client, the SmartNIC operates as a general NIC. The server runs the Lighttpd [26] 1.4.32 web server, ported to FlexServer. The client generates HTTP requests using *dperf* [28], a network load testing tool.

Unless otherwise specified, the experiments are conducted with the following default configurations: (1) The content objects consist of 140,000 files of 1,536 KB (205 GB total), stored across 10 NVMe SSDs. (2) Content requests follow a Zipf distribution with an alpha parameter of 0.8. (3) Both the server and client use 8 CPU cores, with the server configured with four memory channels. (4) Caching is enabled on the SmartNIC, with a cache size of 300 MB.

B. Experimental Results

Effect of asynchronous reads. The first experiment evaluates the file transmission performances of FlexServer with SmartNIC offloading, Linux *sendfile()*, and *mtcp_write()*, and IO-TCP (We use the throughput value reported in [4] due to different settings of using two SmartNICs in IO-TCP). The experiment was run by varying the number of host CPUs from 1 to 8. Figure 4 shows that FlexServer achieves 199 Gbps with 1 CPU and scales to 351 Gbps with 8 CPUs, reaching 94% of the 400 Gbps Ethernet NIC's maximum throughput (374 Gbps in our measurements). In contrast, the maximum throughput at 8 CPUs is only 222 Gbps for Linux

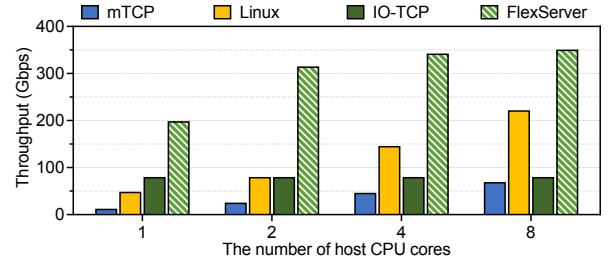


Figure 4: Throughput of FlexServer, mTCP, Linux, and IO-TCP.

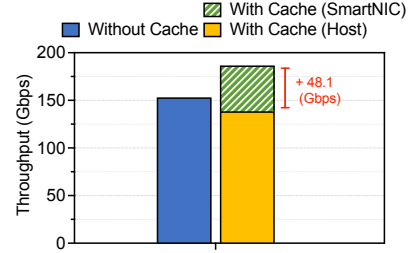


Figure 5: Throughput of FlexServer using one CPU core without cache, and with cache.

sendfile() and 69 Gbps for *mtcp_write()*. The throughput of IO-TCP is about 80 Gbps, regardless of the number of host CPU cores.

Linux *sendfile()* suffers from blocking I/O when file pages are not in memory, leading to lower performance. *mtcp_write()* is inefficient due to frequent data copies and application blocking during file reads. IO-TCP faces high ARM CPU overhead from POSIX disk I/O and underutilized host cores due to static offloading. FlexServer addresses these issues with asynchronous request batching, block-based file read buffers, and SG-DMA for efficient data transmission.

Effect of caching. We evaluate the effectiveness of SmartNIC caching by setting the server's number of memory channels to 2. In the default configuration with 4 memory channels, the host has ample memory bandwidth, making the effect of caching less pronounced. Figure 5 shows the throughput of FlexServer with and without caching when we limit the number of host CPU cores to one. When only one core is used, caching increases the throughput by 33.4 Gbps compared to FlexServer without caching.

C. Microbenchmarks

SSD scalability. In this experiment, we look into how the number of SSDs affects the performance of FlexServer by varying it to 1,2,4,8,10. The results are illustrated in Figure 6. As we increase the number of SSDs, the throughput increases, but it stabilizes at 8 (341 Gbps). The increase in throughput up to 8 SSDs means that the SSD read speed is the bottleneck for the FlexServer throughput under 8 SSDs. When we test with *fio* [29], an I/O performance tester, the maximum read throughput of one SSD is 54 Gbps. Therefore, FlexServer supports up to 6 SSDs at full speed.

Effect of *io_uring* modes. As we discussed in Section II-B, *io_uring* provides multiple modes. We compare the performances of FlexServer by varying the file read mechanism among the default mode and completion polling mode of *io_uring* and POSIX file read in Figure 7. The default mode and completion polling mode of *io_uring* (both 339 Gbps) perform significantly better than using POSIX file reads (207 Gbps). The completion polling mode achieves a similar throughput to the default mode, but one more CPU core must be allocated for polling. Therefore, the default mode performs

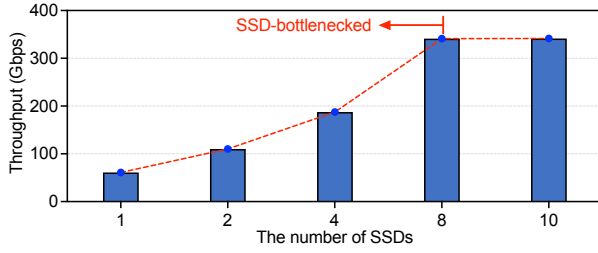


Figure 6: Throughput of FlexServer with different number of SSDs.

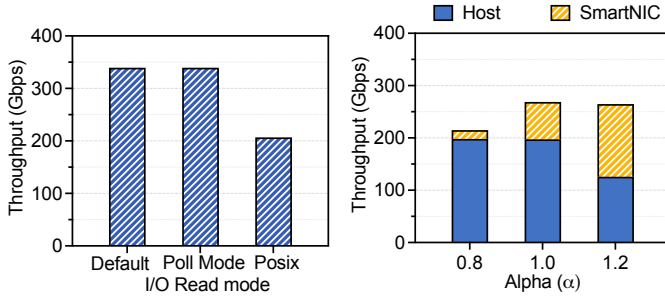


Figure 7: Throughput of FlexServer with default, poll mode of *io_uring*, and POSIX IO.

Figure 8: Throughput of FlexServer with different alpha values of Zipf distribution in 2 channel memory, 4 host CPU core settings.

better than the completion polling mode in terms of CPU efficiency, so we use the default mode for other experiments.

Different request distributions. We evaluate the effect of the different distributions of the requests by varying the alpha (α) parameter of the Zipf request distribution in Figure 8. We run the experiment on a 2-channel memory and 4 CPU core setup because the effect of caching is more pronounced. As we increase the value of alpha, the throughput by SmartNIC caching increases since more file objects can be served by SmartNIC cache. However, the total throughput for the alpha of 1.2 doesn't increase over the alpha of 1.0 because even though the SmartNIC throughput is larger, the throughput by the host is reduced at 1.2.

Overheads analysis. Our analysis of FlexServer's CPU cycle consumptions through *perf* [30], a Linux profiling tool, reveals that the overhead due to memory copy is removed in FlexServer, and the system calls due to file reads take up only 8.5% of the total CPU cycles as shown in Table II. As a result, the web server can utilize more CPU cycles, leading to higher performance. On the other hand, Linux and mTCP consume a large portion of CPU cycles for file I/O system calls and memory copying (67.6% and 30.1%, respectively).

V. RELATED WORK

SmartNICs have been used in a variety of applications to offload tasks that would otherwise be handled by the host CPU. iPipe [13] is an actor-based offloading framework for distributed applications. λ -NIC [14] is for offloading lambdas, small, fine-grained custom programs, to SmartNICs in a serverless computing. KV-Direct [15] is a key-value store system that uses FPGA-based SmartNICs to extend RDMA primitives to provide direct access to the host's main memory. SmartNICs such as [17], [18] support the acceleration of mobile access and core networks. LineFS [19] is a distributed file system with the support of client-local persistent memory that offloads CPU-intensive tasks into SmartNICs. AccelTCP [21], FlexTOE [22], and IO-TCP [4] are techniques designed to accelerate TCP operations

Table II: CPU cycle consumption comparison of FlexServer, Linux, and mTCP (Unit: %).

	Network	Application	File IO System Calls	Memory Copy	Etc.
FlexServer	44.1	20.2	8.5	0	27.2
Linux	9.6	3.3	67.6	0	19.5
mTCP	52	3	25	5.1	15

using SmartNICs. AccelTCP offloads TCP operations such as connection setup, teardown, and packet relay between spliced connections to the SmartNIC [21]. FlexTOE, on the other hand, decomposes the TCP data path into a data-parallel pipeline of processing modules, using NPU-based SmartNICs [22]. IO-TCP offloads both disk I/O and TCP packet transfers to SmartNICs, improving the performance of online content delivery [4]. In contrast, FlexServer differs from these SmartNIC-based TCP offloading schemes by focusing on optimizing I/O operations through advanced asynchronous I/O APIs. It caches popular content and offloads file read operations to the SmartNIC, resulting in significantly higher throughput.

VI. DISCUSSION

Caching in the host. The host typically has a large amount of memory, so caching files on the host and using it as an L2 cache could improve performance. However, our experiments show that this approach provides only marginal improvement. We hypothesize that host caching can reduce CPU cycles for file reads, but not for packet transmission. Since file reads are already efficient due to the use of *io_uring*, host caching does not offer significant benefits. As a result, we do not include the results of host caching in our evaluation.

Achieving line-rate performance. While FlexServer significantly outperforms existing solutions, its maximum throughput is still lower than the line rate. We have confirmed that there are unused resources in CPU cycles, PCIe bandwidth, and memory bandwidth, and the total SSD read speed exceeds the line rate. We believe that further optimizations are needed, particularly for low-level DPDK-related operations. We plan to address this in future work.

VII. CONCLUSION

In this paper, we presented FlexServer, a high-performance content server that leverages SmartNIC technology. FlexServer offloads disk read and packet transfer operations to the SmartNIC, while utilizing the advanced asynchronous I/O mechanism, *io_uring*, to minimize the latency and overhead of disk read operations. To address the inefficiency of duplicate transmission for popular content, FlexServer introduces a caching mechanism at the SmartNIC. Additionally, the multi-type buffer in the control plane ensures TCP consistency. We implemented FlexServer using the mTCP user-level TCP stack and an NVIDIA Bluefield 3 SmartNIC with 400 Gbps Ethernet ports. Through evaluations, we demonstrated that FlexServer delivers significantly higher throughput performance compared to existing solutions, particularly enhancing throughput via SmartNIC offloading when host resources are constrained.

REFERENCES

- [1] "2023 global internet phenomena report." <https://www.sandvine.com/global-internet-phenomena-report-2023>.
- [2] I. Marinos, R. N. Watson, M. Handley, and R. R. Stewart, "Disk! crypt! net: rethinking the stack for high-performance video streaming," in *Proceedings of the Conference of the ACM Special Interest Group on Data Communication*, pp. 211–224, 2017.
- [3] "Intel data direct i/o technology." <https://www.intel.com/content/www/us/en/io/data-direct-i-o-technology.html>.

- [4] T. Kim, D. M. Ng, J. Gong, Y. Kwon, M. Yu, and K. Park, "Rearchitecting the {TCP} stack for {I/O-Offloaded} content delivery," in 20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23), pp. 275–292, 2023.
- [5] E. Jeong, S. Wood, M. Jamshed, H. Jeong, S. Ihm, D. Han, and K. Park, "mtcp: a highly scalable user-level {TCP} stack for multicore systems," in 11th {USENIX} Symposium on Networked Systems Design and Implementation ({NSDI} 14), pp. 489–502, 2014.
- [6] "What is a content delivery network (cdn)?" <https://www.ibm.com/topics/content-delivery-networks>.
- [7] J. Lemon, "Kqueue-a generic and scalable event notification facility," in USENIX Annual Technical Conference, FREENIX Track, pp. 141–153, 2001.
- [8] G. Banga, J. C. Mogul, P. Druschel, et al., "A scalable and explicit event delivery mechanism for unix.," in USENIX Annual Technical Conference, General Track, pp. 253–265, 1999.
- [9] "aio(7) — linux manual page." <https://man7.org/linux/man-pages/man7/aio.7.html>.
- [10] "io_uring." https://man.archlinux.org/man/io_uring.7.en.
- [11] D. Didona, J. Pfefferle, N. Ioannou, B. Metzler, and A. Trivedi, "Understanding modern storage apis: a systematic study of libaio, spdk, and io_uring," in Proceedings of the 15th ACM International Conference on Systems and Storage, pp. 120–127, 2022.
- [12] Z. Ren and A. Trivedi, "Performance characterization of modern storage stacks: Posix i/o, libaio, spdk, and io_uring," in Proceedings of the 3rd Workshop on Challenges and Opportunities of Efficient and Performant Storage Systems, pp. 35–45, 2023.
- [13] M. Liu, T. Cui, H. Schuh, A. Krishnamurthy, S. Peter, and K. Gupta, "Offloading distributed applications onto smartnics using ipipe," in Proceedings of the ACM Special Interest Group on Data Communication, pp. 318–333, 2019.
- [14] S. Choi, M. Shahbaz, B. Prabhakar, and M. Rosenblum, "λ-nic: Interactive serverless compute on programmable smartnics," in 2020 IEEE 40th International Conference on Distributed Computing Systems (ICDCS), pp. 67–77, IEEE, 2020.
- [15] B. Li, Z. Ruan, W. Xiao, Y. Lu, Y. Xiong, A. Putnam, E. Chen, and L. Zhang, "Kv-direct: High-performance in-memory key-value store with programmable nic," in Proceedings of the 26th Symposium on Operating Systems Principles, pp. 137–152, 2017.
- [16] S. Sun, R. Zhang, M. Yan, and J. Wu, "Skv: A smartnic-offloaded distributed key-value store," in 2022 IEEE International Conference on Cluster Computing (CLUSTER), pp. 1–11, IEEE, 2022.
- [17] "N5013/n5014 fpga smartnic." <https://www.silicom-usa.com/pr/4g-5g-products/4g-5g-adapters/n5013-n5014-fpga-smartnic-intel-stratix-based/>.
- [18] "Xilinx 5g telco accelerator cards." <https://www.xilinx.com/publications/presentations/xilinx-5g-telco-accelerator-cards.pdf>.
- [19] J. Kim, I. Jang, W. Reda, J. Im, M. Canini, D. Kostić, Y. Kwon, S. Peter, and E. Witchel, "Linefs: Efficient smartnic offload of a distributed file system with pipeline parallelism," in Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles, pp. 756–771, 2021.
- [20] J. Min, M. Liu, T. Chugh, C. Zhao, A. Wei, I. H. Doh, and A. Krishnamurthy, "Gimbal: enabling multi-tenant storage disaggregation on smartnic jbofs," in Proceedings of the 2021 ACM SIGCOMM 2021 Conference, pp. 106–122, 2021.
- [21] Y. Moon, S. Lee, M. A. Jamshed, and K. Park, "{AccelTCP}: Accelerating network applications with stateful {TCP} offloading," in 17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20), pp. 77–92, 2020.
- [22] R. Shashidhara, T. Stamler, A. Kaufmann, and S. Peter, "{FlexTOE}: Flexible {TCP} offload with {Fine-Grained} parallelism," in 19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22), pp. 87–102, 2022.
- [23] "sendfile(2) — linux manual page." <https://man7.org/linux/man-pages/man2/sendfile.2.html>.
- [24] "Embracing externally allocated memory." https://www.dpdk.org/wp-content/uploads/sites/35/2018/12/Yongseok_Koh_Embracing_Externally_Allocated_Memory.pdf.
- [25] A. Blankstein, S. Sen, and M. J. Freedman, "Hyperbolic caching: Flexible caching for web applications," in USENIX Annual Technical Conference, 2017.
- [26] "Lighttpd - fly light." <https://www.lighttpd.net/>.
- [27] "Nvidia bluefield-3 networking platform." <https://resources.nvidia.com/en-us-accelerated-networking-resource-library/datasheet-nvidia-bluefield>.
- [28] "dperf - a 100gbps network load tester." <https://dperf.org/>.
- [29] "axboe/fio: Flexible i/o tester." <https://github.com/axboe/fio>.
- [30] "perf: Linux profiling with performance counters." <https://perfwiki.github.io/main/>.